Using SSL in MySQL Connection

1. The Modern "Auto-Setup" Method
In modern MySQL versions, if the SSL files are missing from the data directory, the server creates them automatically on startup.

1. **Stop the MySQL service:**

```
sudo systemctl stop mysql
```

2. **Navigate to the Data Directory:**

```
cd /var/lib/mysql
```

3. **Remove (or back up) old .pem files:** If you ran the deprecated tool, you likely have old files there. Removing them allows MySQL to generate fresh, compliant ones.

```
sudo rm *.pem
```

4. **Restart MySQL:**

```
sudo systemctl start mysql
```

5. **Verify:** MySQL will now have generated new certificates with the correct ownership (mysql:mysql). You can verify this by checking the directory again:

```
ls -l /var/lib/mysql/*.pem.
```

2. Manual Configuration (Recommended for Production)
For production environments, you usually want the certificates in a specific, secure location (like /etc/mysql/ssl/) rather than the data directory.

1. **Create a secure directory:**

```
sudo mkdir /etc/mysql/ssl
sudo chown mysql:mysql /etc/mysql/ssl
sudo chmod 700 /etc/mysql/ssl
```

2. **Generate certificates using OpenSSL:** Since the helper tool is deprecated, use the industry-standard openssl.

```
# Generate CA
sudo openssl genrsa 2048 > ca-key.pem
sudo openssl req -new -x509 -nodes -days 3650 -key ca-key.pem -out ca.pem -subj "/CN=MySQL_CA"

# Generate Server Key and Certificate, then sign with CA
sudo openssl req -newkey rsa:2048 -days 3650 -nodes -keyout server-key.pem -out server-req.pem -subj "/CN=MySQL_Server"
sudo openssl x509 -req -in server-req.pem -days 3650 -CA ca.pem -CAkey ca-key.pem -set_serial 01 -out server-cert.pem
```

3. **Move and set permissions:**

```
sudo mv *.pem /etc/mysql/ssl/
sudo chown mysql:mysql /etc/mysql/ssl/*.pem
```

3. Update the Ubuntu Configuration

Open the MySQL configuration file:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

Add these lines to point to your new, manually created files:

```
[mysqld]
ssl-ca=/etc/mysql/ssl/ca.pem
ssl-cert=/etc/mysql/ssl/server-cert.pem
ssl-key=/etc/mysql/ssl/server-key.pem
```

4. Verify Encryption Status

Log in to MySQL and run:

```
SHOW VARIABLES LIKE '%ssl%';
```

If have_ssl is YES, you are good to go. To see if your **active** session is using SSL:

```
\s
```

Look for the **SSL:** line in the output. If it shows a cipher (like TLS_AES_256_GCM_SHA384), your connection is encrypted.

4. Force a User to use SSL

Even if SSL is enabled, users might still connect insecurely unless you enforce it at the user level.

When creating a new user:

```
CREATE USER 'secure_user'@'%' IDENTIFIED BY 'password' REQUIRE SSL;
```

For an existing user:

```
ALTER USER 'app_user'@'%' REQUIRE SSL;
```

5. Connecting from the Client

To connect from a remote CLI, the client needs the ca.pem file generated in Step 1.

```
mysql -u secure_user -p -h 1.2.3.4 --ssl-ca=ca.pem --ssl-mode=REQUIRED
```

Mode	Description
DISABLED	No encryption (Unsafe).
PREFERRED	Tries SSL; falls back to unencrypted if SSL fails.
REQUIRED	Encryption is mandatory, but doesn't check *who* the server is.
VERIFY_CA	Connection fails if the Server's CA certificate doesn't match the client's.